LAPORAN PRAKTIKUM

POSTTEST 5

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhammad Azhril Tirtha

2509106097

C1

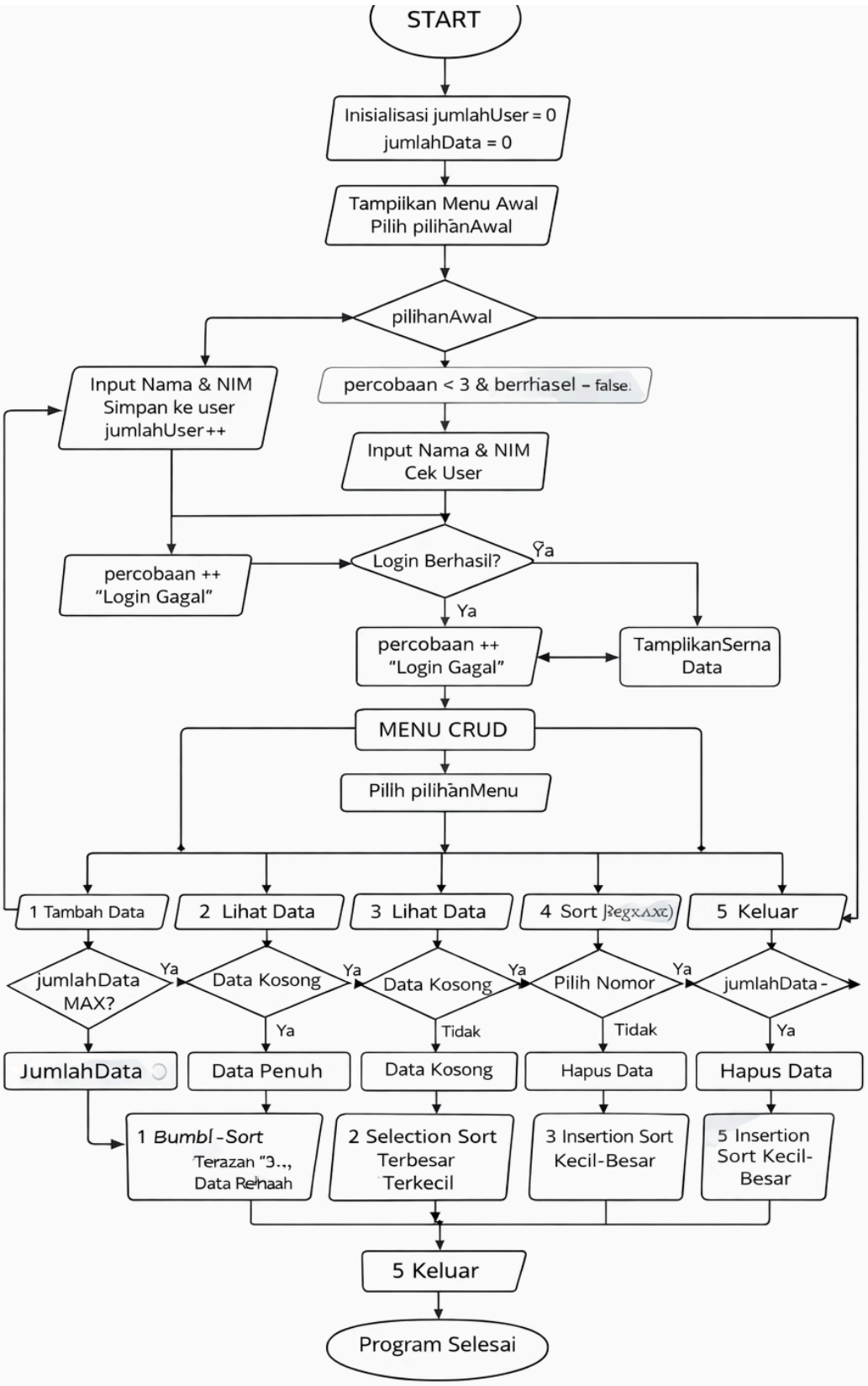
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart


START
Inisialisasi jumlahUser = 0
jumlahData = 0
Tampilkan Menu Awal
Pilih pilihanAwal
pilihanAwal
Input Nama & NIM
Simpan ke user
jumlahUser++
percobaan < 3 & berrhasel – false.
Input Nama & NIM
Cek User
percobaan ++
"Login Gagal"
Login Berhasil?
Ya
Ya
percobaan ++
"Login Gagal"
TamplikanSerna
Data
MENU CRUD
Pillh pilihanMenu
1 Tambah Data
2 Lihat Data
3 Lihat Data
4 Sort
5 Keluar
jumlahData
MAX?
Ya
Data Kosong
Ya
Data Kosong
Ya
Pilih Nomor
Ya
jumlahData -
Ya
Ya
Tidak
Tidak
Ya
JumlahData
Data Penuh
Data Kosong
Hapus Data
Hapus Data
1 Bumbl-Sort
Terazan "3.,
Data Rehaah
2 Selection Sort
Terbesar
Terkecil
3 Insertion Sort
Kecil-Besar
5 Insertion
Sort Kecil-
Besar
5 Keluar
Program Selesai

Gambar 1.1 Flowchart

Penjelasan singkat:

Flowchart dimulai dari **Start**, kemudian program melakukan **inisialisasi jumlah user dan jumlah data**. Setelah itu, program menampilkan **menu awal** yang berisi pilihan register, login, atau keluar.

Jika pengguna memilih **register**, maka pengguna memasukkan nama dan NIM, lalu data disimpan ke dalam array user dan kembali ke menu awal.

Jika memilih **login**, pengguna diminta memasukkan nama dan NIM. Program akan melakukan pengecekan dengan maksimal **3 kali percobaan**. Jika login gagal, percobaan akan bertambah. Jika berhasil, pengguna akan masuk ke **menu CRUD**. Jika gagal 3 kali, program akan berhenti.

Di dalam **menu CRUD**, pengguna dapat memilih:

- **Tambah data** → menambahkan data sayur jika kapasitas belum penuh
- **Lihat data** → menampilkan seluruh data jika tidak kosong
- **Ubah data** → memilih data berdasarkan nomor lalu mengubahnya
- **Hapus data** → menghapus data dan menggeser array
- **Keluar** → kembali ke menu awal

Semua proses dalam menu CRUD akan berulang sampai pengguna memilih keluar. Terakhir, jika pengguna memilih keluar dari menu awal, program akan menampilkan **Program selesai** dan flowchart berakhir di **End**.

2. Deskripsi Singkat Program

Program ini adalah aplikasi berbasis C++ untuk mengelola data toko sayur menggunakan konsep CRUD (Create, Read, Update, Delete). Program diawali dengan fitur register dan login, di mana pengguna harus memasukkan nama dan NIM dengan maksimal 3 kali percobaan. Setelah berhasil login, pengguna dapat menambahkan, melihat, mengubah, dan menghapus data sayur yang disimpan dalam array of struct. Program juga menggunakan fungsi dan prosedur untuk memisahkan tiap proses, serta berjalan secara berulang hingga pengguna memilih keluar.

3. Source Code

```
#include <iostream>
using namespace std;

#define MAX 100
#define MAX_USER 10

struct User {
    string nama;
    string nim;
};

struct Sayur {
    string namaSayur;
    int harga;
    int stok;
};

struct Toko {
    Sayur data;
};

// ================= BUBBLE SORT (NAMA SAYUR ASCENDING) =================
void bubbleSortNama(Toko *dataToko, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
```

```
            if ((dataToko+j)->data.namaSayur >
(dataToko+j+1)->data.namaSayur) {
                swap(*(dataToko+j), *(dataToko+j+1));
            }
        }
    }
}

// ================= SELECTION SORT (HARGA DESCENDING) =================
void selectionSortHarga(Toko *dataToko, int n) {
    for (int i = 0; i < n - 1; i++) {
        int max = i;
        for (int j = i + 1; j < n; j++) {
            if ((dataToko+j)->data.harga > (dataToko+max)->data.harga) {
                max = j;
            }
        }
        swap(*(dataToko+i), *(dataToko+max));
    }
}

// ================= INSERTION SORT (STOK ASCENDING) =================
void insertionSortStok(Toko *dataToko, int n) {
    for (int i = 1; i < n; i++) {
        Toko key = *(dataToko+i);
        int j = i - 1;

        while (j >= 0 && (dataToko+j)->data.stok > key.data.stok) {
            *(dataToko+j+1) = *(dataToko+j);
            j--;
        }
        *(dataToko+j+1) = key;
    }
}

// ================= TAMPIL DATA =================
void tampilData(Toko *dataToko, int jumlahData) {
    if (jumlahData == 0) {
        cout << "Data kosong!\n";
    } else {
        for (int i = 0; i < jumlahData; i++) {
```

```
            cout << i+1 << ". "
                 << (dataToko+i)->data.namaSayur
                 << " | Harga: " << (dataToko+i)->data.harga
                 << " | Stok: " << (dataToko+i)->data.stok << endl;
        }
    }
}

// ================= TAMBAH =================
void tambahData(Toko *dataToko, int *jumlahData) {
    cout << "Nama Sayur: ";
    cin >> (dataToko + *jumlahData)->data.namaSayur;

    cout << "Harga: ";
    cin >> (dataToko + *jumlahData)->data.harga;

    cout << "Stok: ";
    cin >> (dataToko + *jumlahData)->data.stok;

    (*jumlahData)++;
}

// ================= HAPUS =================
void hapusData(Toko *dataToko, int *jumlahData) {
    int nomor;

    tampilData(dataToko, *jumlahData);
    cout << "Pilih nomor: ";
    cin >> nomor;

    if (nomor > 0 && nomor <= *jumlahData) {
        for (int i = nomor-1; i < *jumlahData-1; i++) {
            *(dataToko+i) = *(dataToko+i+1);
        }
        (*jumlahData)--;
    }
}

// ================= LOGIN =================
bool login(User *dataUser, int jumlahUser) {
    string nama, nim;
```

```
    int percobaan = 0;

    while (percobaan < 3) {
        cout << "Nama: ";
        cin >> nama;
        cout << "NIM: ";
        cin >> nim;

        for (int i = 0; i < jumlahUser; i++) {
            if (nama == (dataUser+i)->nama &&
                nim == (dataUser+i)->nim) {
                return true;
            }
        }

        percobaan++;
        cout << "Login gagal\n";
    }

    return false;
}

// ================= MAIN =================
int main() {

    User dataUser[MAX_USER];
    Toko dataToko[MAX];

    int jumlahUser = 0;
    int jumlahData = 0;

    int menuAwal, menu;

    do {
        cout << "\n1. Register\n2. Login\n3. Keluar\nPilih: ";
        cin >> menuAwal;

        if (menuAwal == 1) {
            cout << "Nama: ";
            cin >> dataUser[jumlahUser].nama;
            cout << "NIM: ";
```

```
            cin >> dataUser[jumlahUser].nim;
            jumlahUser++;
        }

        else if (menuAwal == 2) {
            if (login(dataUser, jumlahUser)) {

                do {
                    cout << "\n1. Tambah\n2. Lihat\n3. Hapus\n4. Sort Nama
(ASC)\n5. Sort Harga (DESC)\n6. Sort Stok (ASC)\n7. Keluar\nPilih: ";
                    cin >> menu;

                    switch (menu) {
                        case 1:
                            tambahData(dataToko, &jumlahData);
                            break;
                        case 2:
                            tampilData(dataToko, jumlahData);
                            break;
                        case 3:
                            hapusData(dataToko, &jumlahData);
                            break;
                        case 4:
                            bubbleSortNama(dataToko, jumlahData);
                            cout << "Data berhasil diurutkan (Nama ASC)\n";
                            break;
                        case 5:
                            selectionSortHarga(dataToko, jumlahData);
                            cout << "Data berhasil diurutkan (Harga
DESC)\n";
                            break;
                        case 6:
                            insertionSortStok(dataToko, jumlahData);
                            cout << "Data berhasil diurutkan (Stok ASC)\n";
                            break;
                    }

                } while (menu != 7);
            }
        }
```

```
    } while (menuAwal != 3);

    return 0;
}
```

Gambar 3.1 Source Code

4. Hasil Output

```
=== MENU AWAL ===
1. Register
2. Login
3. Keluar
Pilih: 1
Nama: azhril
NIM: 097
Register berhasil!
```

Gambar 4.1 Output

```
=== MENU AWAL ===
1. Register
2. Login
3. Keluar
Pilih: 2

=== LOGIN ===
Nama: azhril
NIM: 097

=== TOKO SAYUR ===
1. Tambah Data
2. Lihat Data
3. Ubah Data
4. Hapus Data
5. Keluar
Pilih: 1
Nama Sayur: bayam
Harga: 8000
Stok: 20
Data berhasil ditambah!

=== TOKO SAYUR ===
1. Tambah Data
2. Lihat Data
3. Ubah Data
4. Hapus Data
5. Keluar
Pilih: 2

=== DATA SAYUR ===
No 1
Nama  : bayam
Harga : 8000
Stok  : 20
---------------------
```

Gambar 4.2 Output

```
=== TOKO SAYUR ===
1. Tambah Data
2. Lihat Data
3. Ubah Data
4. Hapus Data
5. Keluar
Pilih: 3
1. bayam
Pilih nomor: 1
Nama baru: brokoli
Harga baru: 12000
Stok baru: 50
Data berhasil diubah!

=== TOKO SAYUR ===
1. Tambah Data
2. Lihat Data
3. Ubah Data
4. Hapus Data
5. Keluar
Pilih: 4
1. brokoli
Pilih nomor: 1
Data berhasil dihapus!

=== TOKO SAYUR ===
1. Tambah Data
2. Lihat Data
3. Ubah Data
4. Hapus Data
5. Keluar
Pilih: 5
Keluar menu.
```

Gambar 4.3 Output

```
=== MENU AWAL ===
1. Register
2. Login
3. Keluar
Pilih: 3
Program selesai.
```

Gambar 4.4 Output

```
1. Tambah
2. Lihat
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Keluar
Pilih: 4
Data berhasil diurutkan (Nama ASC)

1. Tambah
2. Lihat
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Keluar
Pilih: 2
1. bayam | Harga: 12000 | Stok: 20
2. bayam | Harga: 13000 | Stok: 30
```

Gambar 4.5 output

```
1. Tambah
2. Lihat
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Keluar
Pilih: 5
Data berhasil diurutkan (Harga DESC)

1. Tambah
2. Lihat
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Keluar
Pilih: 2
1. bayam | Harga: 13000 | Stok: 30
2. bayam | Harga: 12000 | Stok: 20
```

Gambar 4.6 output

```
1. Tambah
2. Lihat
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Keluar
Pilih: 6
Data berhasil diurutkan (Stok ASC)

1. Tambah
2. Lihat
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Keluar
Pilih: 2
1. bayam | Harga: 12000 | Stok: 20
2. bayam | Harga: 13000 | Stok: 30
```

Gambar 4.7 output

5. Langkah-langkah GIT

5.1 GIT Init

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git init
Initialized empty Git repository in D:/praktikum-apl/post-test/post-test-apl-1/.git/
```

Gambar 5.1 Git init

Digunakan untuk membuat repository Git baru di dalam sebuah folder agar folder tersebut dapat dikelola dengan Git.

5.2 GIT Add

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git add .
```

Gambar 5.2 Git add

Digunakan untuk menambahkan file atau perubahan file ke dalam *staging area* sebelum disimpan ke repository.

5.3 GIT Commit

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git commit -m "first commit"
[main (root-commit) 7fca732] first commit
 2 files changed, 76 insertions(+)
 create mode 100644 2509106097-MUHAMMADAZHRILTIRTHA-PT-1.cpp
 create mode 100644 2509106097-MUHAMMADAZHRILTIRTHA-PT-1.exe
```

Gambar 5.3 Git commit

Digunakan untuk menyimpan perubahan yang ada di staging area ke repository Git disertai dengan pesan commit sebagai keterangan perubahan.

5.4 GIT Remote

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git remote add origin https://github.com/muhammadazhriltirtha-debug/praktikum-apl.git
```

Gambar 5.4 Git remote

Digunakan untuk menghubungkan repository lokal dengan repository jarak jauh (misalnya di GitHub), sehingga kode dapat dibagikan atau disinkronkan.

5.5 GIT Push

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 19.95 KiB | 3.99 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/muhammadazhriltirtha-debug/praktikum-apl.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.5 Git push

Digunakan untuk mengirim (upload) commit dari repository lokal ke repository remote, sehingga perubahan dapat tersimpan secara online.